

Sepformer

Phương pháp SepFormer là một **mô hình mạng nơ-ron dựa trên Transformer** được thiết kế để **tách các nguồn âm thanh** (speech separation) từ một tín hiệu hỗn hợp. Ý tưởng cốt lõi của SepFormer là **loại bỏ hoàn toàn các mạng nơ-ron hồi quy (RNN)**, vốn có hạn chế về khả năng song song hóa, và **thay thế bằng các cơ chế attention của Transformer**.

- **Mô hình Không Sử Dụng RNN:** SepFormer được xây dựng dựa trên kiến trúc Transformer, **không sử dụng bất kỳ lớp RNN nào**. Điều này giúp khắc phục các hạn chế về tính tuần tự của RNN, cho phép song song hóa các phép tính và **tăng tốc độ huấn luyện và suy luận**.

- **Kiến trúc Encoder-Masking Network-Decoder:** SepFormer sử dụng mô hình quen thuộc trong tách nguồn âm thanh bao gồm ba thành phần chính:

- **Encoder:** Chuyển đổi tín hiệu âm thanh hỗn hợp từ miền thời gian sang một không gian đặc trưng. Encoder sử dụng một lớp tích chập 1D (Conv1d) và hàm ReLU.
- **Masking Network:** Ước lượng các mask (mặt nạ) tối ưu để tách các nguồn âm thanh từ biểu diễn đã được mã hóa. Phần này sử dụng các khối Transformer để học các phụ thuộc ngắn hạn và dài hạn.
- **Decoder:** Tái tạo lại các nguồn âm thanh đã tách từ biểu diễn đã được masked. Decoder sử dụng một lớp tích chập chuyển vị 1D (ConvTranspose1d).

- **Phương Pháp Dual-Path:** SepFormer sử dụng phương pháp **dual-path** được giới thiệu bởi DPRNN (Dual-Path RNN).

- Phương pháp này chia tín hiệu đầu vào thành các chunk nhỏ hơn.
- Sau đó, nó xử lý các chunk một cách cục bộ (với **Intra-Transformer**) và toàn cục (với **Inter-Transformer**) để học cả phụ thuộc ngắn hạn và dài hạn.
- Cách tiếp cận dual-path này giúp giảm độ phức tạp tính toán của Transformer bằng cách xử lý các chunk nhỏ hơn, thay vì toàn bộ chuỗi tín hiệu.

Encoder

Chức năng chính: Encoder trong SepFormer có vai trò **chuyển đổi tín hiệu âm thanh hỗn hợp** đầu vào từ miền thời gian sang một không gian đặc trưng hữu ích cho việc phân tách nguồn âm thanh. Tín hiệu đầu vào, ký hiệu là $x \in R^T$, chứa âm thanh từ nhiều nguồn khác nhau

- Được triển khai bằng một lớp tích chập 1D (nn.Conv1d) duy nhất, theo sau bởi một hàm kích hoạt ReLU (nn.ReLU). Công thức của quá trình này có thể được viết như sau:

$$h = \text{ReLU}(\text{conv1d}(x)).$$

- conv1d(x): Lớp tích chập 1D sẽ trích xuất các đặc trưng từ tín hiệu đầu vào x.
- ReLU(.): Hàm ReLU (Rectified Linear Unit) được sử dụng để đảm bảo rằng các giá trị đầu ra là không âm

```
class Encoder(nn.Module):
    def __init__(self, L, N):
        super(Encoder, self).__init__()
        self.L = L # Kích thước kernel
        self.N = N # Số lượng kênh đầu ra
        self.Conv1d = nn.Conv1d(in_channels=1, out_channels=N, L
        self.ReLU = nn.ReLU()
    def forward(self, x):
        x = self.Conv1d(x)
        x = self.ReLU(x)
        return x
```

Đầu ra:

Đầu ra của encoder là một biểu diễn $h \in R^{(F \times T')}$, trong đó:

- F là **số lượng kênh đặc trưng (feature channels)**, tương ứng với số lượng bộ lọc trong lớp tích chập. Trong SepFormer, giá trị này được ký hiệu là N .
- T' là **chiều dài thời gian** của biểu diễn đầu ra, có thể ngắn hơn chiều dài tín hiệu đầu vào do sử dụng stride trong quá trình tích chập.

SepFormer Block

Được thiết kế theo cấu trúc "dual-path" (đường dẫn kép), bao gồm hai thành phần chính: intra-Transformer (intraT) và inter-Transformer (interT)

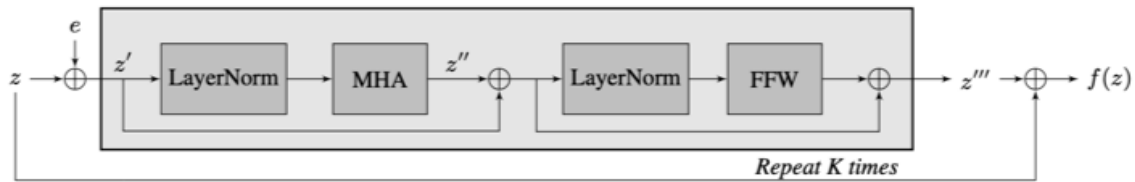
1. Intra-Transformer (IntraT):

- IntraT xử lý các chunk (đoạn) tín hiệu một cách độc lập, tập trung vào việc mô hình hóa các phụ thuộc ngắn hạn bên trong mỗi chunk.
- IntraT nhận đầu vào là các chunk h' đã được chia từ tín hiệu mã hóa h ở quá trình chunking, h' có kích thước $F \times C \times N_c$, trong đó F là số lượng bộ lọc, C là độ dài của chunk, và N_c là số lượng chunk.
- IntraT được áp dụng lên chiều thứ hai của h' , tức là chiều độ dài chunk C , nghĩa là IntraT sẽ xử lý các điểm dữ liệu trong cùng một chunk, tìm kiếm các mối quan hệ ngắn hạn giữa chúng.
- Về cơ bản, IntraT bao gồm nhiều lớp Transformer. Các lớp này sẽ học các phụ thuộc ngắn hạn trong chunk hiện tại
- Trước khi đưa vào các lớp Transformer, positional encoding được thêm vào đầu vào của IntraT. Positional encoding cung cấp thông tin về vị trí tương đối của các phần tử trong chuỗi, giúp các lớp Transformer hiểu được thứ tự của các điểm dữ liệu.

2. Inter-Transformer (InterT):

- Sau khi IntraT xử lý xong, các chiều cuối cùng của đầu ra (tức là C và N_c) sẽ được hoán vị.
- InterT sau đó được áp dụng để mô hình hóa các phụ thuộc dài hạn giữa các chunk.
- Việc hoán vị chiều cho phép InterT xem xét mối quan hệ giữa các chunk khác nhau, giúp nắm bắt các thông tin phụ thuộc theo thời gian dài hơn.
- Tương tự như IntraT, InterT cũng bao gồm nhiều lớp Transformer.
- Positional encoding cũng được thêm vào đầu vào của InterT.

Cấu trúc của các lớp transformer:



1. **Positional Encoding:** Mã hóa vị trí (Positional Encoding) được cộng vào đầu vào để cung cấp thông tin về vị trí tương đối của các phần tử trong chuỗi.
2. **Layer Normalization:** Lớp chuẩn hóa Layer Normalization được áp dụng để ổn định quá trình huấn luyện.
3. **Multi-Head Attention (MHA):** Cơ chế Multi-Head Attention tính toán sự chú ý giữa các phần tử trong chuỗi đầu vào.
4. **Feed-Forward Network (FFN):** Mạng nơ-ron Feed-Forward (truyền thẳng) được áp dụng để thêm tính phi tuyến vào mô hình.
5. **Residual Connections:** Các kết nối tắt (skip connections) được thêm vào để cải thiện việc lan truyền ngược.

```
class DPTBlock(nn.Module):
    // nHead: Số lượng attention head trong Multi-Head Attention.
    // Local_B: Số lượng lớp Transformer được lặp lại trong cả Intra và Inter.
    def __init__(self, input_size, nHead, Local_B):
        super(DPTBlock, self).__init__()
        self.Local_B = Local_B
        self.intra_PositionalEncoding = Positional_Encoding(d_model)
        self.intra_transformer = nn.ModuleList([])
        for i in range(self.Local_B):
            self.intra_transformer.append(TransformerEncoderLayer(
                d_model, nHead, d_ff, dropout, activation

        self.inter_PositionalEncoding = Positional_Encoding(d_model)
        self.inter_transformer = nn.ModuleList([])
        for i in range(self.Local_B):
            self.inter_transformer.append(TransformerEncoderLayer(
                d_model, nHead, d_ff, dropout, activation
```

```

def forward(self, z):
    B, N, K, P = z.shape

    # intra DPT
    row_z = z.permute(0, 3, 2, 1).contiguous().view(B*P, K,
    row_z1 = self.intra_PositionalEncoding(row_z)
    for i in range(self.Local_B):
        row_z1 = self.intra_transformer[i](row_z1.permute(1,
    row_f = row_z1 + row_z
    row_output = row_f.view(B, P, K, N).permute(0, 3, 2, 1)

    # inter DPT
    col_z = row_output.permute(0, 2, 3, 1).contiguous().view
    col_z1 = self.inter_PositionalEncoding(col_z)
    for i in range(self.Local_B):
        col_z1 = self.inter_transformer[i](col_z1.permute(1,
    col_f = col_z1 + col_z
    col_output = col_f.view(B, K, P, N).permute(0, 3, 1, 2)
    return col_output

```

Masking Network

Đầu vào:

Đầu vào của masking network là đầu ra của encoder (h)

Chuẩn hóa và tuyến tính hóa (Norm & Linear)

- Đầu vào h được chuẩn hóa bằng lớp Layer Normalization
 - Layer Normalization giúp ổn định quá trình huấn luyện và cải thiện hiệu suất của mô hình.
- Sau khi chuẩn hóa, h được đưa qua một lớp tuyến tính (Linear layer).

- Lớp tuyến tính này có tác dụng biến đổi không gian đặc trưng của đầu vào, chuẩn bị cho các bước xử lý tiếp theo.
- Lớp tuyến tính được triển khai bằng nn.Linear trong code.

```
def __init__(self, N, C, H, K, Global_B, Local_B):
    super(Separator, self).__init__()
    self.N = N
    self.C = C
    self.K = K
    self.Global_B = Global_B
    self.Local_B = Local_B

    self.LayerNorm = nn.LayerNorm(self.N) # Lớp Layer Normalization
    self.Linear1 = nn.Linear(in_features=self.N, out_features=self.C)

    ...
    x = self.LayerNorm(x.permute(0, 2, 1).contiguous()) # [B, C, L]
    x = self.Linear1(x).permute(0, 2, 1).contiguous() # [B, L, C]
```

Chunking

- Chuỗi thời gian T' của h được chia thành các đoạn (chunk) có kích thước C .
- Các chunk này có overlap (chồng lấn) 50%.
 - Overlap giúp đảm bảo không mất thông tin khi chia nhỏ chuỗi.
- Việc chia nhỏ giúp giảm độ phức tạp tính toán của Transformer, vì Transformer có độ phức tạp bậc hai theo chiều dài chuỗi đầu vào
- Đầu ra h' được sử dụng làm đầu vào để xử lý ở bước sepformer block

```
def pad_segment(self, input, segment_size):
    # Input feature: (B, N, T)
    batch_size, dim, seq_len = input.shape
    segment_stride = segment_size // 2
    rest = segment_size - (segment_stride + seq_len % segment_size)
```

```

        if rest > 0:
            pad = Variable(torch.zeros(batch_size, dim, rest)).float()
            input = torch.cat([input, pad], 2)
        pad_aux = Variable(torch.zeros(batch_size, dim, segment_size)).float()
        input = torch.cat([pad_aux, input, pad_aux], 2)
        return input, rest

def split_feature(self, input, segment_size):
    # Split features into segments of size segment_size
    # Input feature: (B, N, T)
    input, rest = self.pad_segment(input, segment_size)
    batch_size, dim, seq_len = input.shape
    segment_stride = segment_size // 2
    segments1 = input[:, :, :-segment_stride].contiguous().view(batch_size, dim, segment_size)
    segments2 = input[:, :, segment_stride:].contiguous().view(batch_size, dim, segment_size)
    segments = torch.cat([segments1, segments2], 3).view(batch_size, dim, segment_size * 2)
    return segments, rest

# Chunking
out, gap = self.split_feature(x, self.K) # [B, C, L] => [B, C, L']

```

SepFormer Block

Đã được nêu ở phần trên

PReLU và Tuyến tính hóa (PReLU and Linear Transformation)

- Đầu ra h'' từ SepFormer Block được đưa qua lớp kích hoạt PReLU.
 - PReLU là một dạng hàm kích hoạt cải tiến so với ReLU, cho phép các giá trị âm không bị loại bỏ hoàn toàn, giúp tăng tính biểu diễn của mô hình.
- Sau đó, kết quả được đưa qua một lớp tuyến tính (Linear layer).
 - Lớp tuyến tính này biến đổi không gian đặc trưng để tạo ra h''' có kích thước $(F \times N_s) \times C \times N_c$, trong đó N_s là số lượng nguồn âm thanh (ví dụ: số người nói).

```
# PReLU + Linear
h = self.PReLU(h) # [B*K, P, N] => [B*K, P, N]
h = self.Linear2(h) # [B*K, P, N] => [B*K, P, N]
```

Overlap-Add

- Các chunk đã được xử lý được kết hợp lại bằng phương pháp overlap-add.
 - Overlap-add là một phương pháp để tái tạo chuỗi thời gian từ các chunk chồng lấn.

```
def merge_feature(self, input, rest):

    # 将分段的特征合并成完整的话语
    # 输入特征: (B, N, L, K)

    batch_size, dim, segment_size, _ = input.shape
    segment_stride = segment_size // 2
    input = input.transpose(2, 3).contiguous().view(batch_size, dim, segment_size)

    input1 = input[:, :, :, :segment_size].contiguous().view(batch_size, dim, segment_size)
    input2 = input[:, :, :, segment_size:].contiguous().view(batch_size, dim, segment_size)

    output = input1 + input2

    if rest > 0:
        output = output[:, :, :-rest]

    return output.contiguous() # B, N, T
```

Feed-Forward và ReLU

- `h'''` được đưa qua hai lớp feed-forward (FFW).
 - Các lớp FFW được sử dụng để tăng khả năng biểu diễn của mô hình.

- Cuối cùng, một lớp kích hoạt ReLU được áp dụng để tạo ra mask mk cho mỗi nguồn âm thanh.
 - mk có kích thước $F \times T'$ và được sử dụng để tách các nguồn âm thanh.
 - ReLU đảm bảo các giá trị mask không âm.

```
def __init__(self, d_model, nhead, dropout=0):
    super(TransformerEncoderLayer, self).__init__()
    self.LayerNorm1 = nn.LayerNorm(normalized_shape=d_model)
    self.self_attn = MultiheadAttention(d_model, nhead, dropout=
    self.Dropout1 = nn.Dropout(p=dropout)
    self.LayerNorm2 = nn.LayerNorm(normalized_shape=d_model)
    self.FeedForward = nn.Sequential(nn.Linear(d_model, d_model),
                                      nn.ReLU(),
                                      nn.Dropout(p=dropout),
                                      nn.Linear(d_model*2*2, d_model))
    self.Dropout2 = nn.Dropout(p=dropout)

def forward(self, z):
    z1 = self.LayerNorm1(z)
    z2 = self.self_attn(z1, z1, z1, attn_mask=None, key_padding_
    z3 = self.Dropout1(z2) + z
    z4 = self.LayerNorm2(z3)
    z5 = self.Dropout2(self.FeedForward(z4)) + z3
    return z5
```

Tạo mask (Mask generation)

Áp dụng hàm kích hoạt ReLU lên đầu ra của một số lớp tích chập (convolutional layers) và các cơ chế điều khiển (gating mechanisms) để tạo ra kết quả cuối cùng

```
B, _, K, S = out.shape
out = out.view(B, -1, self.C, K, S).permute(0, 2, 1, 3,
out = out.view(B*self.C, -1, K, S)
out = self.merge_feature(out, gap) # [B*C, N, K, S] ->
```

```

        // mask generation
        out = F.relu(self.output(out)*self.output_gate(out))
        out = F.relu(out)

    return out

```

Decoder

Có vai trò chuyển đổi các đặc trưng đã được mã hóa (encoder output) và kết quả ước tính mặt nạ (masking network output) trở lại thành tín hiệu âm thanh ở miền thời gian

Sử dụng một lớp tích chập chuyển vị (transposed convolution layer). Lớp tích chập chuyển vị này có cùng kích thước kernel và stride với lớp tích chập trong encoder.

$$\hat{s}_k = \text{conv1d-transpose}(m_k * h),$$

- $\hat{s}_k$: là tín hiệu nguồn âm thanh thứ k đã được tách (separated source k).
- `conv1d-transpose()`: là lớp tích chập chuyển vị 1D.
- m_k : là mặt nạ (mask) ước tính cho nguồn âm thanh thứ k.
- h : là đầu ra của encoder, biểu diễn các đặc trưng đã được mã hóa của tín hiệu đầu vào¹.

Mục đích của decoder là:

1. **Áp dụng Mặt nạ:** Nhân từng phần tử của mặt nạ m_k với đầu ra của encoder h . Phép nhân này sẽ làm nổi bật các đặc trưng trong h có liên quan đến nguồn âm thanh thứ k, trong khi giảm thiểu các đặc trưng không liên quan.
2. **Tái tạo Tín hiệu:** Sử dụng lớp tích chập chuyển vị để biến đổi các đặc trưng đã được điều chỉnh bởi mặt nạ trở lại thành dạng sóng âm thanh ở miền thời gian³. Lớp tích chập chuyển vị có tác dụng ngược lại so với lớp tích chập trong encoder, giúp khôi phục lại tín hiệu âm thanh từ các đặc trưng đã được mã hóa.

```

class Decoder(nn.Module):
    def __init__(self, L, N):
        super(Decoder, self).__init__()
        self.L = L # Kernel size
        self.N = N # Number of filters in autoencoder
        self.ConvTranspose1d = nn.ConvTranspose1d(in_channels=N,

    def forward(self, x):
        x = self.ConvTranspose1d(x)
        return x

```